

VI LCD w Input Capture

Contreras R Orlando^{1,2*} and Lara H Kevin^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al464376](mailto:al348390@edu.uaa.mx)@edu.uaa.mx;

Abstract

The objective of this lab practice is develop a code to count the RPM(revolutions per minute) of a motor encoder using the input capture timer mode in the micro. The input capture timer mode is important to handle counters by hardware on systems that requires this application

Keywords: ARM, C, API, LCD, Driver, Encoder,

1 Introduction

Implementing hardware solutions in micro controllers is an smart way to improve latency in real time systems, this is like-a-FPGA solution in embedded system. To leverage this in a optimal way, we use multiple hardware solutions like Timers, Counters, ADC, DAC and multiple interruptions.

2 Objectives

- Configure a timer as an external pulse counter.
- Configure a periodic timer ISR to sample counts.
- Calculate RPM from pulses and show values on the LCD using the 4-bit library.
- Provide functions to write strings and integer to display the values.
- Produce a modular, reusable API (LCD handle).

3 Method

3.1 Pseudo-code

Here is the pseudo-code that represents the algorithm, it is also included the configuration as masks or

Algorithm 1 Read updated pulse count from TIM4

Input: *tempCount*: previous counter value**Output:** Updated counter value

```
1  contPulses(tempCount)  curCount  $\leftarrow$  TIM4.CNT  if curCount  $\neq$  tempCount
    then
2  |   tempCount  $\leftarrow$  curCount
3  return tempCount
```

Algorithm 2 Main program to measure and display RPM

Input: None**Output:** Displayed RPM on LCD

```
4  main()  config()// Initialize system configuration
5  LCD_Init4B()// Initialize LCD in 4-bit mode
6  LCD_WriteINT(127)// Test LCD by writing integer 127
7  lastCont  $\leftarrow$  TIM4.CNT
8  while True do
9  |   lastCont  $\leftarrow$  contPulses(lastCont)
10 |   while TIM3.SR  $\wedge$  TIM_SR_UIF do
11 |       RPM  $\leftarrow$  lastCont  $\times$  6// Compute RPM from pulses
12 |       LCD_Clear()  LCD_WriteINT(RPM)  LCD_WriteString(" RPM")
        lastCont  $\leftarrow$  0  TIM3.SR  $\leftarrow$  TIM3.SR  $\wedge$   $\neg$ TIM_SR_UIF      // Clear
        update flag
```

Algorithm 3 Timer configuration for RPM measurement

Input: None**Output:** Configured timers TIM3 and TIM4

```
13 confTIMER()  TIM3.CR1  $\leftarrow$  TIM3.CR1  $\vee$  (1  $\ll$  0)  TIM3.ARR  $\leftarrow$  15999
    TIM3.PSC  $\leftarrow$  999// Configure 100 ms period
14  TIM4.ARR  $\leftarrow$  0xFFFF  TIM4.PSC  $\leftarrow$  1  TIM4.CCMR1  $\leftarrow$  TIM4.CCMR1  $\vee$  (1
     $\ll$  0)// Enable Input Capture channel 1
15  TIM4.CCMR1  $\leftarrow$  TIM4.CCMR1  $\vee$  (3  $\ll$  4)// Enable filter N=8
16  TIM4.CR1  $\leftarrow$  TIM4.CR1  $\vee$  (0x81  $\ll$  0)// Enable autoreload + counter
17  TIM4.CCER  $\leftarrow$  TIM4.CCER  $\vee$  (1  $\ll$  0)// Enable capture channel
18  TIM4.SMCR  $\leftarrow$  TIM4.SMCR  $\vee$  ((7  $\ll$  0)  $\vee$  (5  $\ll$  4))  TIM4.EGR  $\leftarrow$  TIM4.EGR  $\vee$ 
    (1  $\ll$  0)
```

4 Results and Discussion

4.1 Code

Here is the code that represents the previous pseudo-code

Code: main.c

```
#include "conf.h"
```

```

int main(void){
    config();
    LCD_Init4B();
    LCD_WriteINT(127);
    uint16_t lastCont = TIM4->CNT;
    while(1){
        lastCont = contPulses(lastCont);
    while (TIM3->SR & TIM_SR_UIF)
    {
        /*lastCont*60/10 = lastCont*6/1*/
        uint16_t RPM = lastCont*6;
        LCD_Clear();
        LCD_WriteINT(RPM);
        LCD_WriteString(" RPM");
        lastCont = 0;
        TIM3->SR &= ~TIM_SR_UIF;
    }
}

```

4.2 Wiring Diagram

Here is our circuit implementation; we use the pin PB6 as an Alternate Function to use the Input capture embedded in the micro. [Fig 1]

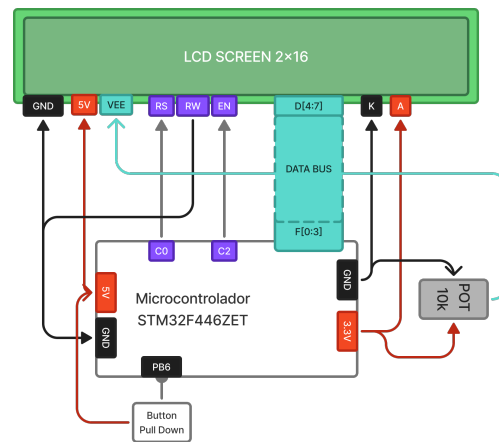


Fig. 1: Wiring Diagram

Q1: Why prefer a hardware counter to software interrupt counting? .

Because improves the latency and avoids to keep the micro-controller busy on a dumb task.

Q2: How to handle very low or very high RPM ranges?

We could handle by implementing a recursive algorithm to decode each digit of the counter to show it in the LCD

Q3: How to guarantee accurate measurement if sample period differs from 1 s?
It's given by a proportion due to different period, if the period is bigger, It requires to divide those period to the samples taken in a second.

5 Conclusion

In this practice, we could observe the functionality of input capture timer mode, which is very useful to create counters integrated by hardware, on systems where the time in each count is valuable and where we need a code execution time independent counter. The RPM were captured with a lot of precision displaying it in a LCD 2X16.